SINGAPORE UNIVERSITY OF SOCIAL SCIENCES

ICT162

Examination - July Semester 2023

Object Oriented Programming

Thursday, 23 November 2023 04:00 pm - 06:00 pm

Time allowed: 2 hours

INSTRUCTIONS TO STUDENTS:

1. This examination contains **4** questions and comprises **10** printed pages (including cover page).
2. You must answer **4** questions.
3. All answers must be written in the answer book.
4. This is **Closed-book** examination.

At the end of the examination.
Please ensure that you have written your examination number on each answer book used.

Failure to do so will mean that your work cannot be identified.
If you have used more than one answer book, please tie them together with the string provided.

THE UNIVERSITY RESERVES THE RIGHT NOT TO MARK YOUR SCRIPT IF YOU FAIL TO FOLLOW THESE INSTRUCTIONS.

ICT162 Copyright © 2023 Singapore University of Social Sciences (SUSS)

(Full marks: 100)

Question 1

In Pizza House, delivery partners are employed to deliver pizza orders to customers. There are 2 types of delivery partners: full-time and part-time. Delivery partners earn their pay based on the number of rounds they deliver in a month. Fees for delivery per round is $32. See Figure Q1-1 for the fees structure for delivery partners.

Delivery Partner	**Number of Rounds Delivered (r)**	
	<= 30	> 30
Full-time	basic salary + r * f	basic salary + r * f
Part-time	r * f	30 * f + (r-30) * (f*1.85)
		where f represents the fees per round.

Figure Q1-1

The class diagram in Figure Q1-2 shows specifications of the delivery partners.

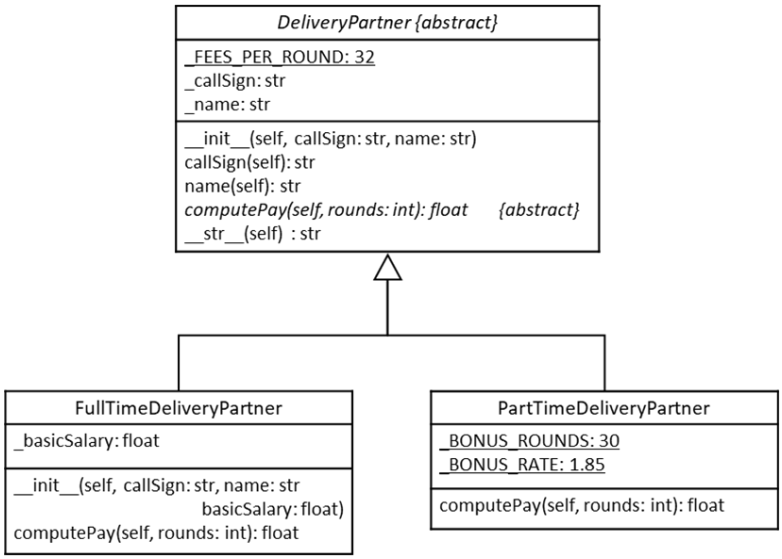


Figure Q1-2

Question 1a

Implement the abstract class – DeliveryPartner.

- _FEES_PER_ROUND: defines the fees earned delivering a round.
- It has 2 instance variables:
  - _callSign (str): the preferred alias or code for this delivery partner. E.g., "DP-007", "Maverick"
  - _name (str): name of the delivery partner.
- Constructor initialises these 2 instance variables: _callSign and _name.
- Getter methods for callSign and name. Use the property decorator.

- One abstract method, computePay() which returns the computed pay for the DeliveryPartner, given the number of rounds delivered.
- The __str__ method returns a string representation of a DeliveryPartner object, which should include the name and callSign. This is an example in the suggested format:

```
Name: Lim Lim Kee      CallSign: DP-007
```

(9 marks)

Question 1b

Write Python class definitions for FullTimeDeliveryPartner and PartTimeDeliveryPartner class.

The FullTimeDeliveryPartner class is a subclass of DeliveryPartner, and it has:

- One additional instance variable:
  - _basicSalary (float): monthly basic salary of a full-time delivery partner.
- Implement the method computePay() based on the fees structure given in Figure Q1-1.

The PartTimeDeliveryPartner class is a subclass of DeliveryPartner, and it has:

- xTwo class variables:
  - _BONUS_ROUNDS: beyond this round number, a part-time delivery partner will be rewarded with a higher fees per round.
  - _BONUS_RATE: representing the bonus rate on the fees per round, should the part-time delivery partner qualify.
- Implement the method computePay() based on the fees structure given in Figure Q1-1.

(10 marks)

Question 1c

Write Python statements to create the appropriate DeliveryPartner objects using the information below, and to print their pay.

Name	**Call Sign**	**Basic Salary**	**Rounds Delivered**
Lim Lim Kee	DP-007	-	50
Tom Yam Kong	Maverick	1,200	40

(6 marks)

Question 2

In Pizza House, an order is placed by a customer. In each order, the customer can select any pizza, of different sizes (S, M, L) and quantity. See a sample order details below, which is self-explanatory:

```
Order ID: 123
Hawaiian Plus - M - $9.59 x 4
Super Supreme - L - $12.59 x 3
Total price: $76.13
```

Study the class diagram in Figure Q2, which showcase the relationships between Order, Customer and Pizza.

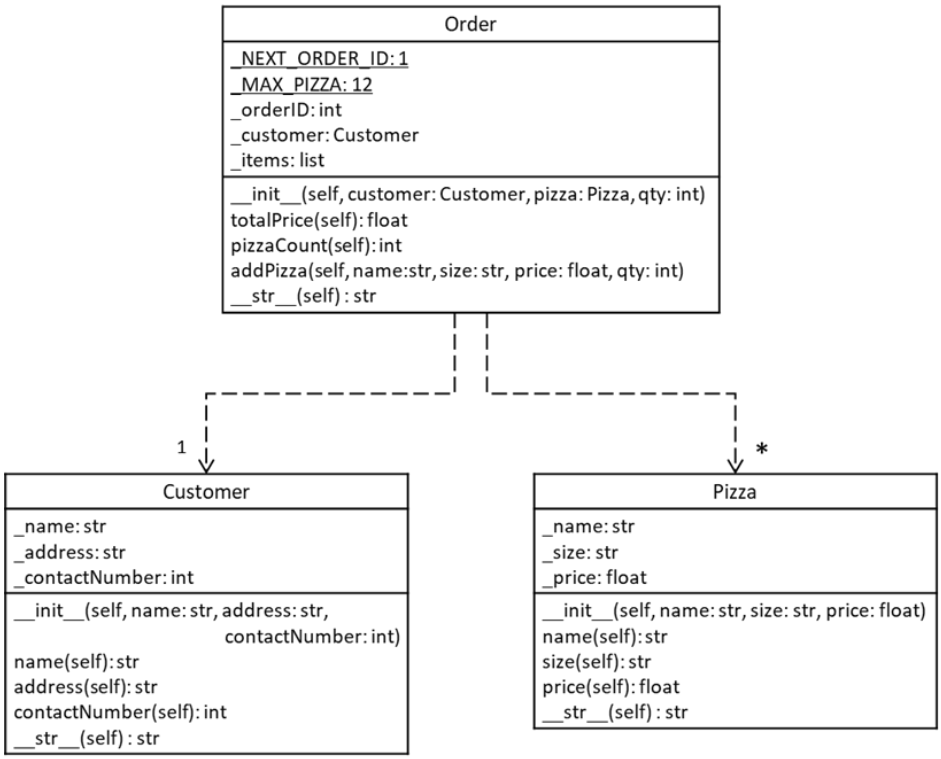


Figure Q2

Assume that the Python codes for Customer and Pizza classes are already provided.

- In Customer class, there are getter methods (property) for instance variables name, address, and contact.
- In Pizza class, there are getter methods (property) for instance variables name, size, and price.

Question 2a

Define the class OrderException, which is a subclass of the Exception class. This class has no additional attribute or method.

(1 mark)

Question 2b

Implement the Order class.

The Order class has:

- Two class variables:
  - _NEXT_ORDER_ID: used to generate the running sequence number for orderID, starting from 1.
  - _MAX_PIZZA: represents the maximum number of pizzas an order can contain. This is also the storage capacity of the vehicle used to deliver pizzas. In a possible scenario, the delivery partner delivers one order up to 12 pizzas.
- Three instance variables:
  - _orderID is the unique ID to identify an order.
  - _customer represents the customer who has placed this order.
  - _items represents the pizza(s) that the customer has ordered. This is implemented using a nested list.
- Constructor that performs the following:
  - Initialise _orderID with the class variable _NEXT_ORDER_ID, the running sequence number .
  - Set _ customer to the given parameter customer.
  - First, set _items to an empty list. Then, create a list with the given parameter pizza and quantity, i.e., [pizza, qty], and add this list into _items.
  - Increment the class variable _NEXT_ORDER_ID, so that the next order will not have duplicate orderID.
- Getter methods for totalPrice and pizzaCount. Use the property decorator.
  - Property totalPrice will return the current price of the order, computed using all the pizzas' price and quantity in this order.
  - Property pizzaCount will return the number of pizzas in this order.
- The purpose of addPizza method is to add more pizza into this order, i.e., into the collection _items.
  - If this addition results in _items having more pizzas than _MAX_PIZZA, raise OrderException with appropriate message.
  - If this order can accommodate this addition, search through _items to see if there is any existing pizza matching the given parameters name and size. (For this TOA, assume the price will be same)
  - If yes, simply add the parameter quantity to the current quantity.
  - If the search is not successful, this addition is for a brand-new pizza. Create the pizza object, and a list containing the pizza object and quantity. Then add into this order, i.e., into the collection _items.
- The __str__ method returns a string representation of an Order object. This is an example in the suggested format:

```
Order ID: 124
Super Supreme - L - $12.59 x 1
Hawaiian Light - M - $8.59 x 2
Total price: $29.77
```

(22 marks)

Question 2c

See following Python codes that create an Order object o1, and has intention to add in more pizzas into this order. Complete the Python codes, with the given instructions starting from line 7. Ensure exceptions are effectively managed.

```
1.  def main():
2.      # Creation of Order
3.      o1 = Order( Customer("Herro Tan", "999A, Sentosa Cove",
4.  99123665),
5.                  Pizza("Hawaiian Plus", "M", 9.59), 1)
6.
7.      # write codes to add the following pizzas in this sequence
8.      # 1. Name: Super Supreme, Size: L, Price: 12.59, Qty: 3
9.      # 2. Name: Hawaiian Plus, Size: M, Price: 9.59, Qty: 3
10.     # 3. Name: Chicken Satay, Size: L, Price: 11.59, Qty: 6
11.     #
12.     # Ensure exceptions are properly handled,
        # such that the order details can be printed after these
    additions
```

(5 marks)

Question 3

Apply the principles of object-oriented programming and develop a program to help Pizza House manage their "DeliveryRound" and implement method to compute the salary of Delivery Partners. Study the class diagram in Figure Q3, which now features the DeliveryRound class.

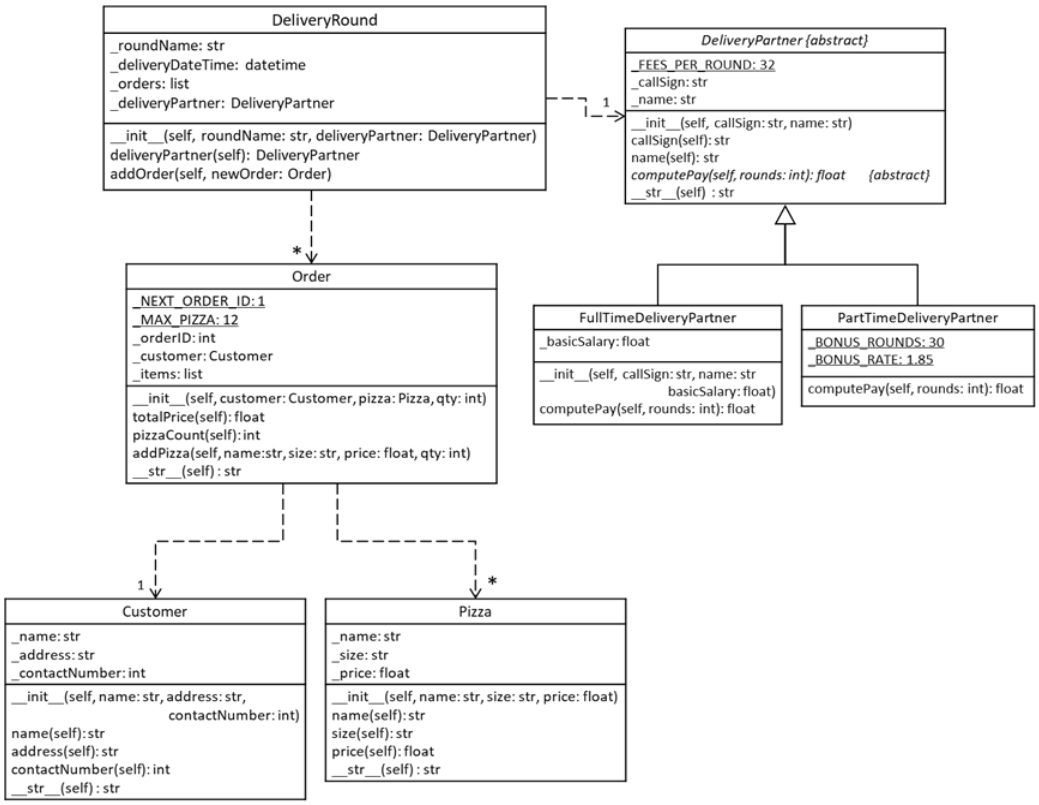


Figure Q3

In Pizza House, each Delivery Partner is given a vehicle that has storage capacity of 12 Pizzas. Hence in a delivery round, the maximum number of pizzas the delivery partner can deliver is 12.

Question 3a

Implement the DeliveryRound class.The DeliveryRound class represents an actual delivery round that the Pizza House will setup for a delivery partner. It has the following attributes:

- Four instance variables:
  - _roundName represents the name given for this delivery round. E.g., "East Coast Path-1".
  - _deliveryDateTime is the start time of the delivery round.
  - _orders is a s list represents all the orders for this delivery round.
  - _deliveryPartner represents the delivery partner doing this delivery round.
- The constructor performs the following:
  - Initialise _roundName and _deliveryPartner with the given parameter roundName and deliveryPartner.
  - Set _deliveryDateTime to the current datetime, and _orders to an empty collection.
- Getter method for deliveryPartner. Use the property decorator.
- The addOrder method has newOrder (an Order object) as parameter.
  - If this addition results in _orders having more pizzas than 12, , i.e., this DeliveryRound has more than 12 pizzas, raise OrderException with appropriate message.
  - Otherwise, add the newOrder into the collection _orders.

(17 marks)

Question 3b

Referring to Figure Q3, for the following pairs of classes, identify between object composition and inheritance, and justify your choices:

- DeliveryRound and Order
- DeliveryRound and FullTimeDeliveryPartner
- DeliveryRound and Pizza

(6 marks)

Question 3c

Referring to Figure Q3, identify **ONE (1)** possible design error in the class diagram (Excluding PizzaHouse class).

(4 marks)

Question 4

KinderWorld has approached SUSS to help develop a Python GUI application to draw shapes of different sizes as part of their educational learning tools for young children. You and your partner are tasked to develop a prototype GUI to draw rectangle only. The GUI allows children to enter the width, height, and to draw the rectangle using either '#' or '@'. The design of the GUI is shown in Figure Q4(a):

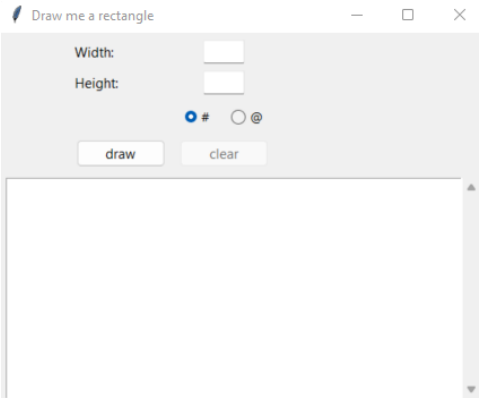


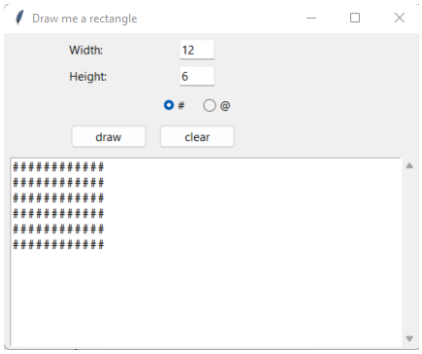
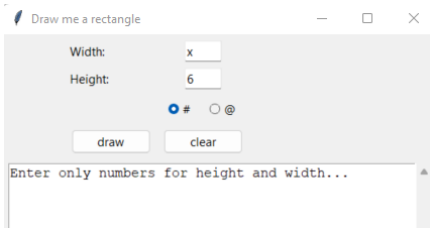
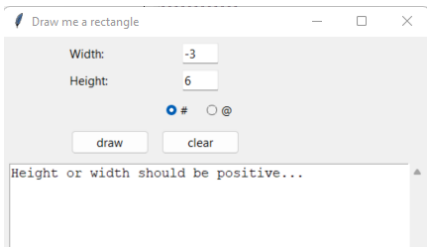
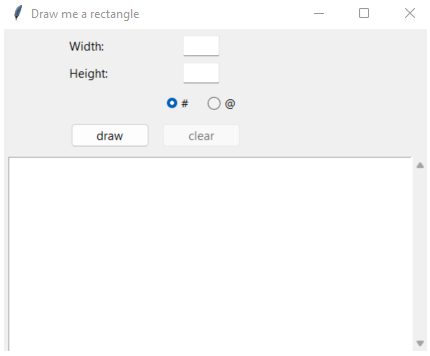
Figure Q4(a)

The following is the partially developed codes from your partner.

```
import tkinter as tk
from tkinter import ttk
from tkinter import scrolledtext

class DrawRectangleGUI:

    def __init__(self):

        self._win = tk.Tk()
        self._win.resizable(True, True)
        self._win.title("Draw me a rectangle")
        self.create_widgets()
        self._win.mainloop()

    def create_widgets(self):
        dataFrame = ttk.Frame(self._win)
        dataFrame.grid(row=0, column=0, padx=8, pady=4)

        width_lbl = ttk.Label(dataFrame, text="Width:")
        width_lbl.grid(row=0, column=0, sticky='W')

        self._width_Ety = ttk.Entry(dataFrame, width=5)
        self._width_Ety.grid(row=0, column=1, padx=4, pady=4)

        height_lbl = ttk.Label(dataFrame, text="Height:")
        height_lbl.grid(row=1, column=0, sticky='W')

        self._height_Ety = ttk.Entry(dataFrame, width=5)
        self._height_Ety.grid(row=1, column=1, padx=4, pady=4)

        radioFrame = ttk.Frame(dataFrame)
        radioFrame.grid(row=2, column=1, padx=8, pady=4)


        self._radValue = tk.IntVar()

        self._radValue.set(0)


        self._hash_rdbtn = ttk.Radiobutton(radioFrame, text='#',

                                    variable=self._radValue, value=0)

        self._at_rdbtn = ttk.Radiobutton(radioFrame, text='@',

                                    variable=self._radValue, value=1)


        self._hash_rdbtn.grid(row=0, column=0, padx=8, pady=4)

        self._at_rdbtn.grid(row=0, column=1, padx=8, pady=4)

        self._draw_btn = ttk.Button(dataFrame, text="draw")
        self._draw_btn.grid(row=3, column=0, padx=4, pady=4)

        self._clear_btn = ttk.Button(dataFrame, text="clear")
        self._clear_btn.config(state=tk.DISABLED)
        self._clear_btn.grid(row=3, column=1, padx=4, pady=4)

        outputFrame = ttk.Frame(self._win)
        outputFrame.grid(row=1, column=0, padx=8, pady=4, columnspan=2)

        self._scrol_stxt = scrolledtext.ScrolledText(outputFrame,
                                                     width=50, height=12,
                                                     wrap=tk.WORD)
        self._scrol_stxt.grid(column=0, row=0, sticky='WE')
```

```
        self._scrol_stxt.config(state=tk.DISABLED)
        self._width_Ety.focus()

    def draw(self):
        pass    # missing codes to draw the rectangle

    def clear(self):
        pass    # missing codes to clear and reset the GUI
```

Write statements that will implement the event handlers for 'draw' and 'clear' buttons. Below are demonstrations of the DrawRectangleGUI:

	When children enter width, height, select '#' and click the "draw" button, your program will draw a rectangle based on the given width and height, using the symbol '#'. "clear" button is also enabled whenever the ScrolledText is populated with any text.
 	Your program should display error messages in the scrolledText when either of the width and height data entry contains: • empty or no data value • non-numeric values • negative numbers
	When "clear" button is clicked, your program should: • clear the ScrolledText • clear the 2 text fields • disable the "clear" button

Submit changes required in the given codes above and codes for draw() and clear() functions.

(20 marks)

----- END OF PAPER -----